Decision Trees

University of South Florida

Group: GGGamers

Group Members

Alexander Bugar
Hector Colon
Liam Dzanan
Seth Wisniewski

Program Description

Our program reads a decision tree from a text file and builds it using linked nodes. It saves a report showing the tree, its properties, and the internal and leaf nodes in preorder. The user can then look up a node by its position to see its content and related nodes. The included files contain decisions about buying a car and choosing an investment.

Instructions to Run on the Student Cluster

Extract the source folder and open a terminal in `Project 3`. With CMake and a C++17 compiler installed, run:

```
cmake -S . -B build
cmake --build build
./build/decision_tree --plain
```

You can also compile and run directly with:

```
g++ -std=c++17 main.cpp LinkedTree.cpp -o decision_tree
./decision_tree
```

Keep the tree files in the folder where you run the program, or enter the full path to a file.

Instructions to Interact

Press Enter after each input.

1. **Choose a file:** Enter `tree-car.txt`, `tree-investment.txt`, or the path to another valid tree file.
2. **Read the report:** Open `about_tree.txt` in the folder where you ran the program. Each run replaces the previous report.
3. **Explore a node:** Enter its preorder position from the file or report. The program shows the node's content, one ancestor, one descendant, and one sibling. It prints `None` when that relative does not exist.
4. **Exit:** Enter `exit` to end the program. An invalid position or other input displays an error and lets you try again.

Implementation Explanation

Each node in `LinkedTree` stores its position, edge label, content, and pointers to its parent, first child, and next sibling. Linking siblings together allows a node to have more than two children. The program sorts the file records by preorder position, then uses their levels to connect each node to its parent.

Recursive functions print the tree, list the nodes, and calculate its properties. The height counts edges on the longest path from the root to a leaf. A binary tree has at most two children per node. It is proper if every internal node has two children, perfect if it is also full at every level, and balanced if the two subtree heights differ by at most one at every node. The destructor deletes the nodes when the program ends.

Included Files

The folder includes the source code, two sample tree files, and `RUN.txt`. The core tree uses only the C++ standard library. The optional interface is in `ui/` and reuses the FTXUI library included with Project 2. For a separate folder without that library, add `-DDECISION_TREE_UI=OFF` to the CMake configuration command.

Tree Interface

Omit `-plain` to open the keyboard menu. Choose a file, then select a walkthrough or browse the nodes. Use the arrow keys and Enter to pick an answer, including any labels besides Yes and No. Left arrow goes back, `r` restarts, and Esc returns to the menu. The colored diagram follows the selected node and adjusts to the terminal size. Ctrl+arrows scroll the map, and `c` centers it again.

Distribution of Work

Alexander Bugar	**25%**
Hector Colon	**25%**
Liam Dzanan	**25%**
Seth Wisniewski	**25%**

Academic Integrity Statement

We understand that there will be no tolerance towards academic dishonesty, and that cheating will lead to an academic referral. We are aware of the identified behaviors that are considered violations of the academic standards for Undergraduate and Graduate students per USF policy.